

Chương 4:

DEADLOCK

ThS. GVC Tô Oai Hùng

1

1

Nội Dung

1. Đặc điểm sử dụng tài nguyên của các tiến trình.
2. Tình trạng deadlock.
3. Giải pháp xử lý.

ThS. GVC Tô Oai Hùng

2

2

4.1 Đặc Điểm Sử Dụng Tài Nguyên của Các Tiến Trình

- ❖ Trong môi trường multiprogramming nhiều tiến trình có thể yêu cầu cùng một tài nguyên để có thể thực thi.
- ❖ Có hai loại tài nguyên:
 - Preemptable: tài nguyên có thể chia sẻ giữa các tiến trình (vd: bộ nhớ).
 - Nonpreemptable: tài nguyên không thể chia sẻ giữa các tiến trình (vd: đầu ghi đĩa).
- ❖ Các bước tiến trình sử dụng tài nguyên :
 - Yêu cầu tài nguyên (*request*).
 - Sử dụng tài nguyên (*use*).
 - Giải phóng (trả) tài nguyên (*release*).

ThS. GVC Tô Oai Hùng

3

3

Đặc Điểm Sử Dụng Tài Nguyên của Các Tiến Trình

- ❖ Khi tiến trình yêu cầu tài nguyên:
 - Nếu tài nguyên có sẵn: Sử dụng.
 - Nếu tài nguyên không có sẵn: Chuyển sang trạng thái chờ.
- ❖ Tài nguyên mà các tiến trình yêu cầu có thể là một tài nguyên không thể chia sẻ.
- ❖ Các tiến trình chờ tài nguyên có thể sẽ không bao giờ thay đổi lại trạng thái được vì các tài nguyên mà nó yêu cầu đang bị giữ bởi các tiến trình khác → tắc nghẽn (*deadlock*).
- ❖ *Deadlock là tình trạng khi hệ thống tồn tại một tập hợp các tiến trình bị khóa, trong đó*

ThS. GVC Tô Oai Hùng

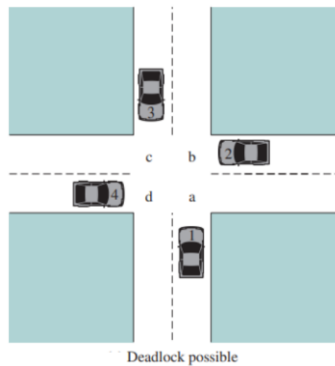
4

4

Đặc Điểm Sử Dụng Tài Nguyên của Các Tiến Trình

mỗi tiến trình đều đang chờ một sự kiện mà chỉ một tiến trình khác có thể tạo ra, và kết quả là không một tiến trình nào có thể hoàn thành.

❖ Ví dụ: Tình trạng kẹt xe.



ThS. GVC Tô Oai Hùng

5

5

4.2. Tình Trạng Deadlock

1. Điều kiện xảy ra deadlock.
2. Đồ thị cấp phát tài nguyên.

ThS. GVC Tô Oai Hùng

6

6

Điều Kiện Xảy Ra Deadlock

- ❖ Ngăn chặn lẫn nhau (*mutual exclusion*): Một tài nguyên bị chiếm bởi một tiến trình, và không tiến trình nào khác có thể sử dụng tài nguyên này.
- ❖ Giữ và đợi (*hold and wait*): Một tiến trình đang giữ ít nhất một tài nguyên và chờ một số tài nguyên khác đang bị một tiến trình khác chiếm giữ.
- ❖ Không có đặc quyền (*no preemption*): Không thể thu hồi tài nguyên đã cấp cho tiến trình mà phải chờ tiến trình trả lại tài nguyên sau khi đã sử dụng xong.
- ❖ Chờ đợi vòng tròn (*circular wait* - là một trường hợp của Giữ và đợi): Một tập tiến trình $\{P_0, P_1, \dots, P_n\}$:

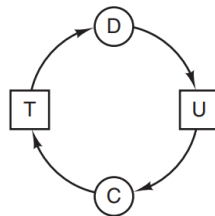
ThS. GVC Tô Oai Hùng

7

7

Điều Kiện Xảy Ra Deadlock

- P0 chờ một tài nguyên do P1 chiếm giữ.
- P1 chờ một tài nguyên khác do P2 chiếm giữ, ...
- Pn-1 chờ tài nguyên do Pn chiếm giữ.
- Pn chờ tài nguyên do P0 chiếm giữ.



- ❖ Khi có đủ 4 điều kiện này thì deadlock xảy ra. Nếu thiếu một trong 4 điều kiện trên thì không có deadlock.

ThS. GVC Tô Oai Hùng

8

8

Đồ Thị Cấp Phát Tài Nguyên

- ❖ Đồ thị cấp phát tài nguyên (*Resource allocation graph*) dùng để mô tả một cách chính xác tình trạng bế tắc.
- ❖ Là một đồ thị có hướng $G=(V, E)$ với V là tập đỉnh, E là tập các cạnh.
- ❖ V được chia thành hai tập con:
 - $P= \{P_0, P_1, ..., P_n\}$ là tập các tiến trình trong hệ thống.
 - $R= \{R_0, R_1, ..., R_m\}$ là tập các loại tài nguyên trong hệ thống thỏa mãn $P \cup R = E$ và $P \cap R = \emptyset$.

ThS. GVC Tô Oai Hùng

9

9

Đồ Thị Cấp Phát Tài Nguyên

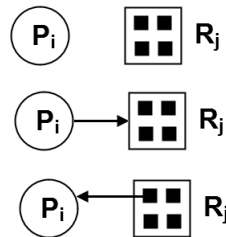
- ❖ Một cạnh có hướng từ P_i tới R_j (ký hiệu $P_i \rightarrow R_j$): Tiến trình P_i chờ tài nguyên R_j - gọi là cạnh yêu cầu.
- ❖ Một cạnh có hướng từ R_j tới P_i (ký hiệu $R_j \rightarrow P_i$): Tài nguyên R_j đã được cấp phát cho tiến trình P_i - gọi là cạnh gán.
- ❖ Biểu diễn:
 - P_i : Hình tròn.
 - R_j : Hình chữ nhật. R_j có thể có nhiều thể hiện $\rightarrow$ biểu diễn mỗi thể hiện là một chấm nằm trong hình vuông.
 - Một cạnh yêu cầu trở tới chỉ một hình vuông R_j .
 - Một cạnh gán gán tới một trong các dấu chấm trong hình vuông.

ThS. GVC Tô Oai Hùng

10

10

Đồ Thị Cấp Phát Tài Nguyên



- ❖ Khi P_i yêu cầu một thể hiện R_j , một cạnh yêu cầu được chèn vào đồ thị cấp phát tài nguyên.
- ❖ Khi yêu cầu được đáp ứng, cạnh yêu cầu được truyền tới cạnh gán.
- ❖ Khi quá trình không còn cần truy xuất tới tài nguyên, nó giải phóng tài nguyên → cạnh gán bị xoá.

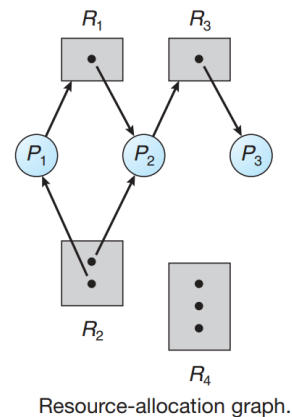
ThS. GVC Tô Oai Hùng

11

11

Đồ Thị Cấp Phát Tài Nguyên

- ❖ Ví dụ: các tập P , R , và E :
 - $P = \{P_1, P_2, P_3\}$
 - $R = \{R_1, R_2, R_3, R_4\}$
 - $E = \{P_1 \rightarrow R_1, P_2 \rightarrow R_3, R_1 \rightarrow P_2, R_2 \rightarrow P_2, R_2 \rightarrow P_1, R_3 \rightarrow P_3\}$
- ❖ Trạng thái tiến trình:
 - P_1 đang giữ một thể hiện của R_2 và đang chờ một thể hiện của R_1 .
 - P_2 đang giữ một thể hiện của R_1 và R_2 và đang chờ một thể hiện của R_3 .
 - P_3 đang giữ một thể hiện của R_3 .



ThS. GVC Tô Oai Hùng

12

12

Đồ Thị Cấp Phát Tài Nguyên

- ❖ Nếu đồ thị không chứa chu trình: Không có deadlock.
- ❖ Nếu đồ thị có chứa chu trình: Có thể có deadlock.
- ❖ Nếu một chu trình bao gồm chỉ một tập hợp các loại tài nguyên, mỗi loại tài nguyên chỉ có một thể hiện:
 - Có deadlock.
 - Mỗi tiến trình chứa trong chu trình bị deadlock.
 - Một chu trình trong đồ thị là điều kiện cần và đủ để tồn tại deadlock.
- ❖ Nếu mỗi loại tài nguyên có nhiều thể hiện:

ThS. GVC Tô Oai Hùng

13

13

Đồ Thị Cấp Phát Tài Nguyên

- Chu trình không có deadlock.
- Một chu trình trong đồ thị là điều kiện cần nhưng chưa đủ để tồn tại deadlock.

ThS. GVC Tô Oai Hùng

14

14

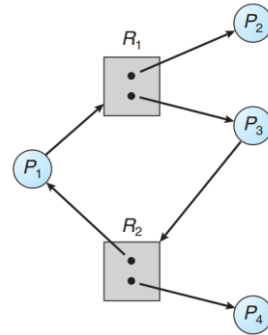
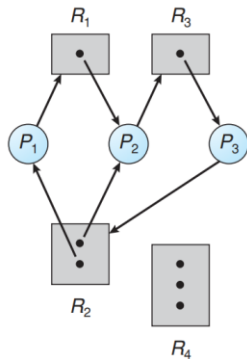
Đồ Thị Cấp Phát Tài Nguyên

P3 yêu cầu một thể hiện của R2
→ có deadlock:

- $P_1 \rightarrow R_1 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$
- $P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_2$

Có chu trình nhưng không có deadlock:

- $P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$



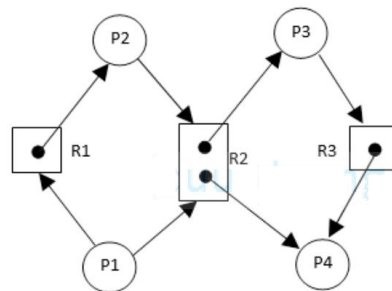
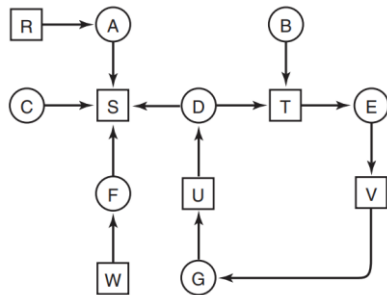
ThS. GVC Tô Oai Hùng

15

15

Đồ Thị Cấp Phát Tài Nguyên

❖ Xét đồ thị cấp phát tài nguyên sau, cho biết hệ thống có xảy ra tình trạng deadlock hay không?



ThS. GVC Tô Oai Hùng

16

16

4.3 Giải Pháp Xử Lý Deadlock

1. Không quan tâm: Có thể bỏ qua, xem như deadlock không bao giờ xảy ra trong hệ thống.
2. Ngăn chặn deadlock.
3. Tránh xảy ra deadlock.
4. Phát hiện tình trạng deadlock và khôi phục.

ThS. GVC Tô Oai Hùng

17

17

4.3.1 Ngăn Chặn Deadlock

- ❖ Ngăn chặn một trong bốn điều kiện làm xảy ra deadlock:
 - Loại trừ lẫn nhau (*mutual exclusion*).
 - Giữ và chờ cấp thêm tài nguyên (*hold and wait*).
 - Không đòi lại tài nguyên từ quá trình đang giữ chúng (*no preemption*).
 - Tồn tại chu trình trong đồ thị cấp phát tài nguyên (*circular wait*).

ThS. GVC Tô Oai Hùng

18

18

Ngăn Chặn Mutual Exclusion

- ❖ Đảm bảo hệ thống không có các tài nguyên không thể chia sẻ: Gần như không tránh được.

ThS. GVC Tô Oai Hùng

19

19

Ngăn Chặn Hold and Wait

- ❖ Đảm bảo khi một tiến trình yêu cầu tài nguyên, nó không giữ bất cứ tài nguyên nào khác:
 - Bắt buộc mỗi tiến trình phải yêu cầu toàn bộ tài nguyên cần thiết một lần, nếu có đủ tài nguyên hệ thống sẽ cấp phát, nếu không đủ tài nguyên, tiến trình sẽ bị block.
 - Khi yêu cầu tài nguyên, tiến trình không được giữ một tài nguyên khác, nếu có thì phải trả tài nguyên đang giữ trước khi yêu cầu.
 - ❖ Hạn chế :
 - Hiệu quả sử dụng tài nguyên rất thấp.
- Có khả năng starvation.

ThS. GVC Tô Oai Hùng

20

20

Ngăn Chặn Không Có Đặc Quyền

- ❖ Cách 1: Nếu một tiến trình đang giữ một số tài nguyên và yêu cầu tài nguyên không có sẵn thì tiến trình phải trả tất cả tài nguyên hiện đang giữ.
- ❖ Cách 2: Nếu tài nguyên mà một tiến trình yêu cầu không có sẵn:
 - Nếu các tài nguyên đó đang được cấp phát tới các tiến trình khác đang chờ tài nguyên bổ sung → thu hồi lại → cấp cho tiến trình đang yêu cầu.
 - Nếu tài nguyên đó đang được cấp phát tới một tiến trình không đợi tài nguyên, tiến trình đang yêu cầu phải chờ và một

ThS. GVC Tô Oai Hùng

21

21

Ngăn Chặn Không Có Đặc Quyền

số tài nguyên nó đang giữ có thể được đòi lại nếu có tiến trình khác yêu cầu.

- ❖ Phương pháp này thường được áp dụng cho các tài nguyên mà trạng thái của nó có thể được lưu lại dễ dàng và phục hồi lại sau đó, như các thanh ghi CPU và không gian bộ nhớ.
- ❖ Nó thường không thể áp dụng cho các tài nguyên như máy in và đĩa từ.

ThS. GVC Tô Oai Hùng

22

22

Ngăn Chặn Tạo Chu Trình Trong Đồ Thị Cấp Phát Tài Nguyên

- ❖ Cấp phát tài nguyên theo một thứ tự phân cấp như sau:
 - Gọi $R = \{R_1, R_2, \dots, R_m\}$ là tập hợp loại tài nguyên.
 - Đánh số thứ tự cho mỗi loại tài nguyên bằng cách định nghĩa hàm ánh xạ một-một $F: R \rightarrow N$ (N là tập hợp các số tự nhiên)
 - Ví dụ: $F(\text{ổ băng từ}) = 1$, $F(\text{đĩa từ}) = 5$, $F(\text{máy in}) = 12$.
 - Một tiến trình đang giữ tài nguyên R_i thì chỉ được yêu cầu tài nguyên R_j nếu: $F(R_j) > F(R_i)$

ThS. GVC Tô Oai Hùng

23

23

4.3.2 Tránh Deadlock

- ❖ Sử dụng thuật toán cấp phát tài nguyên với đầu vào là các thông tin yêu cầu cấp phát tài nguyên của tiến trình để quyết định cấp phát tài nguyên sao cho deadlock không xảy ra.
- ❖ Có nhiều thuật toán tránh deadlock.
- ❖ Thuật toán đơn giản:
 - Biết trước số thể hiện của mỗi loại tài nguyên mà mỗi tiến trình sẽ sử dụng.
 - → Hệ thống sẽ có đủ thông tin để xây dựng thuật toán cấp phát không gây ra tắc nghẽn.

ThS. GVC Tô Oai Hùng

24

24

Tránh Deadlock

- ❖ Mục tiêu các thuật toán là kiểm tra trạng thái cấp phát tài nguyên "động" để đảm bảo điều kiện tạo chu trình chờ không xảy ra.
- ❖ Trạng thái cấp phát tài nguyên được xác định bởi:
 - Số lượng tài nguyên rồi.
 - Số lượng tài nguyên đã cấp phát cho các tiến trình.
 - Số lượng tối đa tài nguyên mà mỗi tiến trình yêu cầu.
- ❖ Hai thuật toán:
 - Thuật toán đồ thị cấp phát tài nguyên.
 - Thuật toán banker.

ThS. GVC Tô Oai Hùng

25

25

Trạng Thái An Toàn

- ❖ Một trạng thái cấp phát tài nguyên được gọi là an toàn nếu hệ thống có thể cấp phát tài nguyên cho các tiến trình theo một thứ tự nào đó mà vẫn tránh được deadlock.
- ❖ Nói cách khác, một hệ thống ở trong trạng thái an toàn nếu và chỉ nếu ở đó tồn tại một thứ tự an toàn.
- ❖ Thứ tự của các tiến trình $\langle P_1, P_2, \dots, P_n \rangle$ là một thứ tự an toàn khi: Với mỗi thứ tự P_i , các tài nguyên mà P_i yêu cầu vẫn có thể được thỏa mãn bởi:
 - Tài nguyên hiện có của hệ thống + các tài nguyên được giữ bởi tất cả P_j khác, với $j < i$.

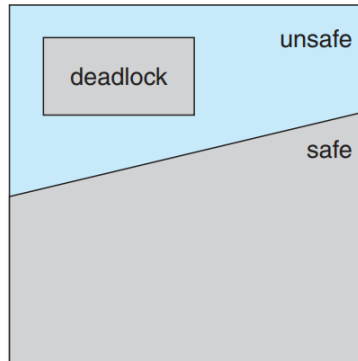
ThS. GVC Tô Oai Hùng

26

26

Trạng Thái An Toàn

- ❖ Trạng thái an toàn không là trạng thái tắc nghẽn.
- ❖ Trạng thái tắc nghẽn là trạng thái không an toàn.
- ❖ Trạng thái không an toàn có thể không là trạng thái tắc nghẽn.



ThS. GVC Tô Oai Hùng

Safe, unsafe, and deadlocked state spaces.

27

27

Trạng Thái An Toàn

- ❖ Ví dụ: Xét một hệ thống có 12 băng từ và 3 tiến trình P0, P1, P2 với các yêu cầu cấp phát:
 - P0 yêu cầu tối đa 10 băng từ.
 - P1 yêu cầu tối đa 4 băng từ.
 - P2 yêu cầu tối đa 9 băng từ.
- ❖ Giả sử tại thời điểm t0:
 - P0 đang giữ 5 băng từ
 - P1 và P2: mỗi tiến trình giữ 2 băng từ.
 - → có 3 băng từ rỗi.

P	Yêu cầu nhiều nhất	Yêu cầu hiện tại
P0	10	5
P1	4	2
P2	9	2

ThS. GVC Tô Oai Hùng

28

28

Trạng Thái An Toàn

- ❖ Tại thời điểm t_0 , hệ thống ở trạng thái an toàn: Thứ tự cấp phát $\langle P_1, P_0, P_2 \rangle$ thỏa mãn điều kiện an toàn.
- ❖ Giả sử ở thời điểm t_1 , P_2 có yêu cầu và được cấp phát 1 băng từ $\rightarrow$ Hệ thống không ở trạng thái an toàn nữa.

ThS. GVC Tô Oai Hùng

29

29

Giải Thuật Đồ Thị Cấp Phát Tài Nguyên

- ❖ Được áp dụng cho các tài nguyên chỉ có 1 thể hiện.
- ❖ Sử dụng đồ thị cấp phát tài nguyên và bổ sung thêm một cạnh báo trước / yêu cầu (*claim edge*).
- ❖ Cạnh báo trước $P_i \rightarrow R_j$ cho biết P_i có thể yêu cầu cấp phát tài nguyên R_j , được biểu diễn trên đồ thị bằng các đường có nét đứt.
- ❖ Các tài nguyên mà tiến trình cần phải được thông báo trước: Tất cả các cạnh báo trước của P_i phải xuất hiện trong đồ thị cấp phát tài nguyên.

ThS. GVC Tô Oai Hùng

30

30

Giải Thuật Đồ Thị Cấp Phát Tài Nguyên

- ❖ Giả sử P_i yêu cầu tài nguyên R_j . Yêu cầu này chỉ có thể được chấp nhận nếu ta chuyển cạnh báo trước $P_i \rightarrow R_j$ thành cạnh cấp phát $R_j \rightarrow P_i$ mà không tạo ra chu trình.
- ❖ Kiểm tra bằng cách sử dụng thuật toán phát hiện chu trình trong đồ thị: Nếu có n tiến trình trong hệ thống, thuật toán phát hiện chu trình có độ phức tạp tính toán $O(n^2)$.
 - Nếu không có chu trình: Trạng thái an toàn $\rightarrow$ cấp phát.
 - Ngược lại: Trạng thái không an toàn.

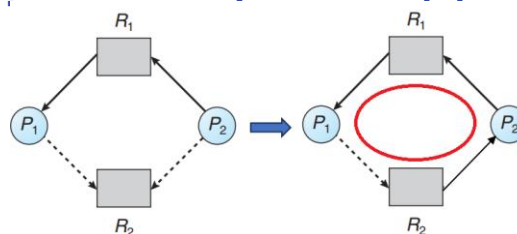
ThS. GVC Tô Oai Hùng

31

31

Giải Thuật Đồ Thị Cấp Phát Tài Nguyên

- ❖ Ví dụ: Giả sử P_2 yêu cầu cấp phát R_2 .



- ❖ Mặc dù R_2 rảnh nhưng chúng ta không thể cấp phát R_2 , vì nếu cấp phát ta sẽ có chu trình trong đồ thị và gây ra chờ vòng $\rightarrow$ Hệ thống ở trạng thái không an toàn.
- ❖ Nhận xét: không thể áp dụng tới hệ thống cấp phát tài nguyên với nhiều thể hiện của mỗi loại tài nguyên.

ThS. GVC Tô Oai Hùng

32

32

Giải Thuật Banker

- ❖ Còn gọi là giải thuật nhà băng.
- ❖ Được dùng cho các hệ thống mà mỗi tài nguyên có nhiều thể hiện, kém hiệu quả hơn thuật toán đồ thị phân phối tài nguyên.
- ❖ Thuật toán banker có thể dùng trong ngân hàng: Không bao giờ cấp phát tài nguyên (tiền) gây nên tình huống sau này không đáp ứng được nhu cầu của tất cả các khách hàng.
- ❖ Khi một tiến trình mới đưa vào hệ thống, hệ thống phải biết tiến trình cần tối đa bao nhiêu thể hiện của mỗi loại tài nguyên để có thể thực thi.

ThS. GVC Tô Oai Hùng

33

33

Giải Thuật Banker

- ❖ Khi tiến trình yêu cầu các tài nguyên, hệ thống phải xác định việc cấp phát các tài nguyên này có đảm bảo hệ thống ở trạng thái an toàn hay không.
 - An toàn: Cấp tài nguyên.
 - Không an toàn: Tiến trình phải chờ cho tới khi đủ tài nguyên.

ThS. GVC Tô Oai Hùng

34

34

Giải Thuật Banker

- ❖ Gọi n là số tiến trình trong hệ thống và m là số loại tài nguyên trong hệ thống, cần có các cấu trúc dữ liệu:
 - Available: Một vector có chiều dài m biểu diễn số lượng tài nguyên sẵn có của mỗi loại.
 - Nếu $Available[j] = k \rightarrow$ có k thể hiện của loại tài nguyên R_j sẵn dùng.
 - Max: Ma trận $n \times m$ biểu diễn số lượng tối đa yêu cầu của mỗi tiến trình đối với mỗi loại tài nguyên.
 - Nếu $Max[i, j] = k \rightarrow P_i$ có thể yêu cầu nhiều nhất k thể hiện của loại tài nguyên R_j .

ThS. GVC Tô Oai Hùng

35

35

Giải Thuật Banker

- Allocation: Ma trận $n \times m$ biểu diễn số lượng tài nguyên của mỗi loại mà mỗi tiến trình đang giữ.
 - Nếu $Allocation[i, j] = k \rightarrow P_i$ hiện được cấp k thể hiện của loại tài nguyên R_j .
- Need: Ma trận $n \times m$ biểu diễn số lượng tài nguyên của mỗi loại mà mỗi tiến trình đang cần thêm.
 - Nếu $Need[i, j] = k \rightarrow$ tiến trình P_i có thể cần thêm k thể hiện của loại tài nguyên R_j để hoàn thành tác vụ của nó.
 - $Need[i, j] = Max[i, j] - Allocation[i, j]$
- ❖ Số lượng và giá trị các biến trên biến đổi theo trạng thái của hệ thống.

ThS. GVC Tô Oai Hùng

36

36

Giải Thuật Banker

- ❖ Qui ước so sánh hai vector: Gọi X và Y là các vector có chiều dài n , ta có:
 - $X \leq Y$ nếu và chỉ nếu $X[i] \leq Y[i]$, $\forall i$.
 - Ví dụ:
 - $X = (0, 3, 2, 1)$
 - $Y = (1, 7, 3, 2)$
 - $\Rightarrow X \leq Y$.

ThS. GVC Tô Oai Hùng

37

37

Giải Thuật Banker

- ❖ Thuật toán trạng thái an toàn:
 1. Với $Work$ và $Finish$ là hai vector độ dài m và n . Khởi tạo:
 - $Work = Available$.
 - $Finish[i] = false$ for $i = 0, 1, \dots, n - 1$.
 2. Tìm i sao cho $Finish[i] == false$ và $Need[i] \leq Work$.
 - Nếu không tìm được i , đến bước 4.
 3. $Work = Work + Allocation[i]$;
 $Finish[i] = true$;
 Chuyển đến bước 2;
 4. Nếu $Finish[i] == true$ $\forall i$ thì hệ thống ở trạng thái an toàn.
- ❖ Độ phức tạp của thuật toán: $O(m.n^2)$.

ThS. GVC Tô Oai Hùng

38

38

Giải Thuật Banker

❖ Thuật toán yêu cầu tài nguyên:

1. Nếu $\text{Request}[i] \leq \text{Need}[i]$, đến bước 2. Ngược lại, thông báo lỗi (không có tài nguyên rồi).
2. Nếu $\text{Request}[i] \leq \text{Available}$, đến bước 3. Ngược lại, P_i phải chờ vì không có tài nguyên
3. Giả sử hệ thống cấp phát các tài nguyên cho tiến trình $P_i \rightarrow$ Cập nhật trạng thái như sau:
 - $\text{Available} = \text{Available} - \text{Request}[i]$.
 - $\text{Allocation}[i] = \text{Allocation}[i] + \text{Request}[i]$.
 - $\text{Need}[i] = \text{Need}[i] - \text{Request}[i]$.

ThS. GVC Tô Oai Hùng

39

39

Giải Thuật Banker

4. Áp dụng thuật toán kiểm tra trạng thái an toàn:
 - Nếu hệ thống ở trạng thái an toàn $\rightarrow$ Cấp phát tài nguyên cho P_i .
 - Ngược lại $\rightarrow P_i$ phải chờ và trạng thái của hệ thống được khôi phục như cũ.

ThS. GVC Tô Oai Hùng

40

40

Giải Thuật Banker

- ❖ Ví dụ, hệ thống gồm:
 - 3 loại tài nguyên A, B, C.
 - A có 10 thể hiện.
 - B có 5 thể hiện.
 - C có 7 thể hiện.
 - 5 tiến trình P0, P1, P2, P3, P4.
 - Giả sử trạng thái của hệ thống tại thời điểm T0:

	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	3	3	2
P1	2	0	0	3	2	2			
P2	3	0	2	9	0	2			
P3	2	1	1	2	2	2			
P4	0	0	2	4	3	3			

41

Giải Thuật Banker

- Tại thời điểm T0, hệ thống có ở trạng thái an toàn hay không?
- Giả sử P1 yêu cầu cấp phát (1,0 2). Yêu cầu này có thể được thỏa mãn hay không?

42

Giải Thuật Banker

❖ Giải:

	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	3	3	2
P1	2	0	0	3	2	2			
P2	3	0	2	9	0	2			
P3	2	1	1	2	2	2			
P4	0	0	2	4	3	3			

■ Ma trận Need = Max – Allocation:

	Need		
	A	B	C
P0	7	4	3
P1	1	2	2
P2	6	0	2
P3	0	1	1
P4	4	3	1

ThS. GVC Tô Oai Hùng

43

43

Giải Thuật Banker

- Work = Available = (3, 3, 2).
- Finish[i] = false, $\forall i$.
- Tìm i:
 - i = 1:
 - Need[1] $\leq$ Work.
 - Work = (2, 0, 0) + (3, 3, 2) = (5, 3, 2).
 - Finish[1] = true.
 - i = 3:
 - Need[3] $\leq$ Work.
 - Work = (5, 3, 2) + (2, 1, 1) = (7, 4, 3).
 - Finish[3] = true.
 - i = 4:
 - Need[4] $\leq$ Work.
 - Work = (7, 4, 3) + (0, 0, 2) = (7, 4, 5).
 - Finish[4] = true.

ThS. GVC Tô Oai Hùng

44

44

Giải Thuật Banker

- $i = 0$:
 $Need[0] \leq Work$.
 $Work = (7, 4, 5) + (0, 1, 0) = (7, 5, 5)$.
 $Finish[0] = true$.
- $i = 2$:
 $Need[2] \leq Work$.
 $Work = (7, 5, 5) + (3, 0, 2) = (10, 5, 7)$.
 $Finish[2] = true$.
- Hệ thống hiện ở trong trạng thái an toàn theo thứ tự $\langle P1, P3, P4, P0, P2 \rangle$.

ThS. GVC Tô Oai Hùng

45

45

Giải Thuật Banker

- Giả sử rằng quá trình $P1$ yêu cầu thêm $(1, 0, 2)$:
 - $Request[1] \leq Available \rightarrow True$.
 - $Request[1] \leq Need[1] \rightarrow True$.
 - Cập nhật trạng thái mới:

	<u>Allocation</u>			<u>Need</u>			<u>Available</u>		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	4	3	2	3	0
P_1	3	0	2	0	2	0			
P_2	3	0	2	6	0	0			
P_3	2	1	1	0	1	1			
P_4	0	0	2	4	3	1			

- Hãy kiểm tra hệ thống bằng thuật toán trạng thái an toàn?

ThS. GVC Tô Oai Hùng

46

46

4.3.3 Phát Hiện và Xử Lý Deadlock

- ❖ Nếu không áp dụng phòng tránh hoặc ngăn chặn deadlock thì hệ thống có thể bị deadlock. Khi đó:
 - Cần có thuật toán kiểm tra trạng thái để xem có deadlock xuất hiện hay không.
 - Thuật toán khôi phục nếu deadlock xảy ra.
- ❖ Cần có giải thuật áp dụng cho hệ thống mà mỗi loại tài nguyên chỉ có một thể hiện và hệ thống mà mỗi loại tài nguyên có nhiều thể hiện

ThS. GVC Tô Oai Hùng

47

47

Tài Nguyên Chỉ Có Một Thể Hiện

- ❖ Sử dụng thuật toán đồ thị chờ (wait-for) có được từ đồ thị cấp phát tài nguyên bằng cách xóa các đỉnh tài nguyên và nối các cung liên quan.
- ❖ $P_i \rightarrow P_j$: P_i đang chờ P_j giải phóng tài nguyên mà P_i cần.
- ❖ $P_i \rightarrow P_j$: tồn tại trong đồ thị chờ nếu và chỉ nếu đồ thị cấp phát tài nguyên tương ứng có hai cung:
 - $P_i \rightarrow R_q$
 - $R_q \rightarrow P_j$
- ❖ Hệ thống có deadlock nếu đồ thị chờ có chu trình.

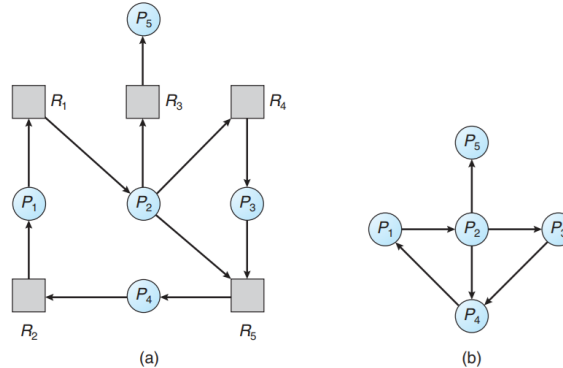
ThS. GVC Tô Oai Hùng

48

48

Tài Nguyên Chỉ Có Một Thẻ Hiện

- ❖ Để phát hiện deadlock:
 - Cần cập nhật đồ thị chờ.
 - Thực hiện định kỳ thuật toán phát hiện chu trình.



ThS. GVC Tô Oai Hùng

49

49

Tài Nguyên Có Nhiều Thẻ Hiện

- ❖ Sử dụng các cấu trúc dữ liệu tương tự như giải thuật Banker:
 - Available: Vector m phần tử biểu diễn số lượng thẻ hiện của mỗi loại tài nguyên.
 - Allocation: Ma trận $n \times m$ biểu diễn số thẻ hiện của mỗi loại tài nguyên đang được cấp phát cho các tiến trình.
 - Request: Ma trận $n \times m$ biểu diễn yêu cầu hiện tại của mỗi tiến trình.
 - Nếu $\text{Request}[i][j] = k \rightarrow$ tiến trình P_i yêu cầu cấp phát k thẻ hiện của tài nguyên R_j .

ThS. GVC Tô Oai Hùng

50

50

Tài Nguyên Có Nhiều Thể Hiện

1. Khởi tạo:

- Work = Available
- For $i = 0, 1, \dots, n-1$
 - Nếu $\text{Allocation}[i] \neq 0$, gán $\text{Finish}[i] = \text{false}$.
 - Ngược lại, gán $\text{Finish}[i] = \text{true}$

2. Tìm i sao cho $\text{Finish}[i] == \text{false}$ và $\text{Request}[i] \leq \text{Work}$. Nếu không tìm thấy i , chuyển đến bước 4.

3. $\text{Work} = \text{Work} + \text{Allocation}[i]$; $\text{Finish}[i] = \text{true}$; $\rightarrow$ chuyển đến bước 2

4. Nếu $\text{Finish}[i] == \text{false}$ với $0 \leq i \leq n-1$ thì hệ thống đang bị tắc nghẽn (và tiến trình P_i đang tắc nghẽn).

❖ Độ phức tạp: $O(m.n^2)$.

51

51

Tài Nguyên Có Nhiều Thể Hiện

❖ Ví dụ: Có 5 tiến trình P_0, P_1, P_2, P_3, P_4 và 3 loại tài nguyên A, B, C.

- A có 7 thể hiện.
- B có 2 thể hiện.
- C có 6 thể hiện.
- Trạng thái cấp phát tài nguyên tại thời điểm T_0 :

	Allocation			Request			Available		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	0	0	0	0	0	0
P_1	2	0	0	2	0	2			
P_2	3	0	3	0	0	0			
P_3	2	1	1	1	0	0			
P_4	0	0	2	0	0	2			

ThS. GVC Tô Oai Hùng

52

52

Tài Nguyên Có Nhiều Thẻ Hiện

- Hệ thống không ở trong trạng thái deadlock (thứ tự $\langle P_0, P_2, P_3, P_4, P_1 \rangle \rightarrow \text{Finish}[i] = \text{true}$ cho với mọi i).
- Giả sử P_2 yêu cầu thêm 1 thẻ hiện của tài nguyên C.
- Ma trận request được sửa lại như sau:

	<u>Request</u>		
	A	B	C
P_0	0	0	0
P_1	2	0	2
P_2	0	0	1
P_3	1	0	0
P_4	0	0	2

- Hệ thống bị deadlock bao gồm các tiến trình P_1, P_2, P_3, P_4 .

ThS. GVC Tô Oai Hùng

53

53

Sử Dụng Giải Thuật Phát Hiện Deadlock

- ❖ Khi nào sử dụng giải thuật phát hiện deadlock? Phụ thuộc vào hai yếu tố:
 - Deadlock có khả năng xảy ra thường xuyên?
 - Bao nhiêu tiến trình sẽ bị ảnh hưởng khi có deadlock?
- ❖ Nếu deadlock xảy ra thường xuyên thì nên sử dụng giải thuật phát hiện deadlock.
- ❖ Sử dụng thuật toán phát hiện:
 - Khi có yêu cầu cấp phát tài nguyên: Tổng tài nguyên CPU.
 - Sử dụng giải thuật trong mỗi giờ hay bất cứ khi nào việc sử dụng CPU rơi xuống thấp hơn 40%.

ThS. GVC Tô Oai Hùng

54

54